

File: aoi_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
import time
from .cnc_controller import GRBLCNCController
from .camera_controller import CameraController
from .position_manager import InspectionPositionManager, InspectionPosition, InspectionSequence

class CNCAOIController:
    """
    Controlador principal que integra o movimento CNC e captura de imagem
    para aplicações de Inspeção Óptica Automatizada (AOI).
    """

    def __init__(self, serial_port=None, camera_interface=None):
        """
        Inicializa o controlador AOI.

        Args:
            serial_port: Porta serial para conexão com a CNC (opcional)
            camera_interface: Interface para a câmera (opcional)
        """
        self.cnc = GRBLCNCController()
        self.camera = CameraController(camera_interface)
        self.position_manager = InspectionPositionManager()
        self.is_running_sequence = False
        self.current_sequence = None
        self.current_position_index = -1

    def connect_cnc(self, port=None, baudrate=115200):
        """Conecta à máquina CNC."""
        return self.cnc.connect(port, baudrate)

    def connect_camera(self, camera_id=0):
        """Conecta à câmera."""
        return self.camera.connect(camera_id)

    def add_inspection_position(self, name, x, y, camera_params=None):
        """
        Adiciona uma posição de inspeção ao gerenciador.

        Args:
            name: Nome descritivo da posição
            x, y: Coordenadas
            camera_params: Parâmetros específicos da câmera (opcional)
        """
        position = InspectionPosition(name, x, y, camera_params)
        self.position_manager.add_position(position)
        return position

    def add_current_position(self, name, camera_params=None):
```

```
"""Adiciona a posição atual como uma posição de inspeção."""
```

```
position = self.cnc.get_current_position()
```

```
return self.add_inspection_position(name, position['x'], position['y'], camera_params)
```

```
def create_sequence(self, name, positions=None):
```

```
"""Cria uma nova sequência de inspeção."""
```

```
sequence = InspectionSequence(name)
```

```
if positions:
```

```
    for pos in positions:
```

```
        sequence.add_position(pos)
```

```
self.position_manager.add_sequence(sequence)
```

```
return sequence
```

```
def run_sequence(self, sequence_name, callback=None):
```

```
"""
```

```
Executa uma sequência de inspeção completa.
```

```
Args:
```

```
    sequence_name: Nome da sequência a executar
```

```
    callback: Função chamada após cada captura com imagem e metadados
```

```
"""
```

```
sequence = self.position_manager.get_sequence(sequence_name)
```

```
if not sequence:
```

```
    raise ValueError(f"Sequência '{sequence_name}' não encontrada")
```

```
self.is_running_sequence = True
```

```
self.current_sequence = sequence
```

```
self.current_position_index = 0
```

```
# Iniciar execução da sequência
```

```
return self._process_next_position(callback)
```

```
def _process_next_position(self, callback=None):
```

```
"""Processa a próxima posição na sequência."""
```

```
if not self.is_running_sequence or self.current_position_index >= len(self.current_sequence.positions):
```

```
    self.is_running_sequence = False
```

```
    return None
```

```
# Obtém a próxima posição
```

```
position = self.current_sequence.positions[self.current_position_index]
```

```
# Move para a posição
```

```
self.cnc.move_to_absolute_position(position.x, position.y)
```

```
# Verifica se chegou na posição
```

```
while self.cnc.is_moving():
```

```
    time.sleep(0.1)
```

```
# Captura a imagem
```

```
image = self.camera.capture(position.camera_params)
```

```
# Prepara dados para callback
result = {
    "position": position,
    "image": image,
    "timestamp": time.time(),
    "sequence": self.current_sequence.name,
    "index": self.current_position_index
}
```

```
# Avança para a próxima posição
self.current_position_index += 1
```

```
# Chama o callback se fornecido
if callback:
    callback(result)
```

```
return result
```

```
def stop_sequence(self):
    """Para a execução da sequência atual."""
    self.is_running_sequence = False
```

```
def save_positions(self, filename):
    """Salva todas as posições e sequências em um arquivo."""
    return self.position_manager.save_to_file(filename)
```

```
def load_positions(self, filename):
    """Carrega posições e sequências de um arquivo."""
    return self.position_manager.load_from_file(filename)
```

File: camera_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
class CameraController:
```

```
    """
```

```
    Classe para controlar uma câmera e capturar imagens.
    Pode ser adaptada para diferentes interfaces de câmera.
```

```
    """
```

```
def __init__(self, camera_interface=None):
```

```
    """
```

```
    Inicializa o controlador de câmera.
```

```
    Args:
```

```
        camera_interface: Interface de câmera personalizada (opcional)
```

```
    """
```

```
self.camera = camera_interface
```

```
self.is_connected = False
```

```
self.last_error = ""
```

```
def connect(self, camera_id=0):
```

```
    """
```

```
    Conecta à câmera.
```

```
    Args:
```

```
        camera_id: ID da câmera (para OpenCV)
```

```
    Returns:
```

```
        bool: True se conectado com sucesso
```

```
    """
```

```
    if self.camera:
```

```
        # Usa a interface fornecida
```

```
        try:
```

```
            self.is_connected = self.camera.connect()
```

```
            return self.is_connected
```

```
        except Exception as e:
```

```
            self.last_error = str(e)
```

```
            return False
```

```
    else:
```

```
        # Implementação padrão com OpenCV
```

```
        try:
```

```
            import cv2
```

```
            self.camera = cv2.VideoCapture(camera_id)
```

```
            self.is_connected = self.camera.isOpened()
```

```
            return self.is_connected
```

```
        except Exception as e:
```

```
            self.last_error = str(e)
```

```
            return False
```

```
def disconnect(self):
```

```
    """Desconecta da câmera."""
```

```
    if not self.is_connected:
```

```
        return
```

```
    try:
```

```
        if hasattr(self.camera, 'release'):
```

```
            self.camera.release()
```

```
        elif hasattr(self.camera, 'disconnect'):
```

```
            self.camera.disconnect()
```

```
    self.is_connected = False
```

```
    except Exception as e:
```

```
        self.last_error = str(e)
```

```
def capture(self, params=None):
```

```
    """
```

```
    Captura uma imagem da câmera.
```

```
    Args:
```

```
        params: Parâmetros específicos da câmera (exposição, etc.)
```

Returns:

A imagem capturada ou None em caso de erro

"""

if not self.is_connected:

return None

try:

Para interface personalizada

if hasattr(self.camera, 'capture'):

return self.camera.capture(params)

Para OpenCV

import cv2

ret, frame = self.camera.read()

if ret:

Aplicar parâmetros se fornecidos

if params:

Exemplo: ajustar brilho/contraste

if 'brightness' in params:

frame = cv2.convertScaleAbs(frame, alpha=params['brightness'])

return frame

else:

self.last_error = "Falha ao capturar imagem"

return None

except Exception as e:

self.last_error = str(e)

return None

def set_parameters(self, **kwargs):

"""

Define parâmetros da câmera.

Args:

**kwargs: Parâmetros específicos da câmera

"""

if not self.is_connected:

return False

try:

Para interface personalizada

if hasattr(self.camera, 'set_parameters'):

return self.camera.set_parameters(**kwargs)

Para OpenCV

import cv2

for key, value in kwargs.items():

if key.lower() == 'exposure':

self.camera.set(cv2.CAP_PROP_EXPOSURE, value)

elif key.lower() == 'focus':

```
        self.camera.set(cv2.CAP_PROP_FOCUS, value)
        # Adicione outros parâmetros conforme necessário
```

```
    return True
```

```
except Exception as e:
    self.last_error = str(e)
    return False
```

File: cnc_controller.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```
import time
import serial
import serial.tools.list_ports
from threading import Thread
from queue import Queue, PriorityQueue
import threading

class GRBLCommunicationThread(Thread):
    """
    Thread para comunicação com o controlador GRBL.
    """

    def __init__(self):
        super().__init__()
        self.daemon = True
        self.serial_port = None
        self.running = False
        self.command_queue = PriorityQueue()
        self.lock = threading.Lock()

        # Eventos simulados
        self.response_received = EventEmitter()
        self.status_update = EventEmitter()
        self.position_update = EventEmitter()
        self.error_message = EventEmitter()

    def set_serial_port(self, serial_port):
        """Define a porta serial para comunicação."""
        self.serial_port = serial_port
        self.running = True

    def run(self):
        """Método principal da thread."""
        while True:
            if not self.running or not self.serial_port:
                time.sleep(0.1)
                continue
```

```

# Processar comandos na fila
try:
    if not self.command_queue.empty():
        priority, command = self.command_queue.get(block=False)
        with self.lock:
            self.serial_port.write((command + "\n").encode())
        self.command_queue.task_done()

# Ler respostas
if self.serial_port.in_waiting:
    with self.lock:
        response = self.serial_port.readline().decode().strip()

    if response:
        self.response_received.emit(response)

# Processar atualizações de status e posição
if response.startswith('<'):
    self.status_update.emit(response)
    self._parse_position(response)
elif response.startswith('error:'):
    self.error_message.emit(response)
except Exception as e:
    self.error_message.emit(str(e))

time.sleep(0.01)

def _parse_position(self, status_string):
    """Analisa a string de status para extrair a posição."""
    try:
        parts = status_string.split('|')
        for part in parts:
            if part.startswith('MPos:') or part.startswith('WPos:'):
                coords = part.split(':')[1].split(',')
                position = {'x': float(coords[0]), 'y': float(coords[1]), 'z': float(coords[2])}
                self.position_update.emit(position)
            break
    except Exception:
        pass

def send_command(self, command, priority=False):
    """
    Envia um comando para a fila.

    Args:
        command: O comando GCODE
        priority: Se True, o comando tem prioridade
    """
    priority_level = 0 if priority else 1
    self.command_queue.put((priority_level, command))

```

```
def disconnect(self):
    """Desconecta a thread."""
    self.running = False
    if self.serial_port and self.serial_port.is_open:
        self.serial_port.close()
    self.serial_port = None
```

```
class EventEmitter:
```

```
    """
```

Sistema de eventos para comunicação assíncrona.
Permite registrar múltiplos callbacks para diferentes eventos.

```
    """
```

```
def __init__(self):
    self.callbacks = {}
    self.default_callbacks = []
```

```
def connect(self, callback, event_name=None):
```

```
    """
```

Conecta um callback a um evento específico ou como callback padrão.

Args:

callback: Função a ser chamada quando o evento ocorrer
event_name: Nome do evento (se None, será adicionado aos callbacks padrão)

```
    """
```

```
if event_name is None:
    self.default_callbacks.append(callback)
else:
    if event_name not in self.callbacks:
        self.callbacks[event_name] = []
    self.callbacks[event_name].append(callback)
```

```
# Retorna uma função que pode ser usada para desconectar este callback
return lambda: self.disconnect(callback, event_name)
```

```
def disconnect(self, callback, event_name=None):
```

```
    """
```

Desconecta um callback específico.

Args:

callback: Callback a ser removido
event_name: Nome do evento do qual remover (se None, remove dos callbacks padrão)

```
    """
```

```
if event_name is None:
    if callback in self.default_callbacks:
        self.default_callbacks.remove(callback)
else:
    if event_name in self.callbacks and callback in self.callbacks[event_name]:
        self.callbacks[event_name].remove(callback)
```

```
def disconnect_all(self, event_name=None):
```

```
"""
```

Desconecta todos os callbacks de um evento específico ou todos os eventos.

Args:

event_name: Nome do evento para limpar (se None, limpa todos os eventos)

```
"""
```

if event_name is None:

self.callbacks.clear()

self.default_callbacks.clear()

else:

if event_name in self.callbacks:

self.callbacks[event_name].clear()

```
def emit(self, *args, event_name=None, **kwargs):
```

```
"""
```

Emita um evento, chamando todos os callbacks registrados.

Args:

*args: Argumentos posicionais para passar aos callbacks

event_name: Nome do evento a ser emitido (se None, apenas callbacks padrão são chamados)

**kwargs: Argumentos nomeados para passar aos callbacks

```
"""
```

Chama todos os callbacks padrão

for callback in self.default_callbacks:

try:

callback(*args, **kwargs)

except Exception as e:

print(f"Erro em callback para evento: {e}")

Se um nome de evento for especificado, chama esses callbacks também

if event_name is not None and event_name in self.callbacks:

for callback in self.callbacks[event_name]:

try:

callback(*args, **kwargs)

except Exception as e:

print(f"Erro em callback para evento '{event_name}': {e}")

```
def has_listeners(self, event_name=None):
```

```
"""
```

Verifica se há callbacks registrados para um evento específico ou para qualquer evento.

Args:

event_name: Nome do evento a verificar (se None, verifica todos)

Returns:

bool: True se houver callbacks registrados, False caso contrário

```
"""
```

if event_name is None:

return bool(self.default_callbacks) or any(bool(callbacks) for callbacks in self.callbacks.values())

else:

return event_name in self.callbacks and bool(self.callbacks[event_name])

```
class GRBLCNCController:
```

```
    """
```

```
    Classe para controlar uma máquina CNC usando GRBL.
```

```
    Adaptada do código existente para ser usada como biblioteca.
```

```
    """
```

```
def __init__(self):
```

```
    """Inicializa o controlador CNC."""
```

```
    self.comm_thread = GRBLCommunicationThread()
```

```
    self.comm_thread.response_received.connect(self._on_response_received)
```

```
    self.comm_thread.status_update.connect(self._on_status_update)
```

```
    self.comm_thread.position_update.connect(self._on_position_update)
```

```
    self.comm_thread.error_message.connect(self._on_error_message)
```

```
    self.is_connected = False
```

```
    self.current_position = {'x': 0, 'y': 0, 'z': 0}
```

```
    self.machine_status = "Disconnected"
```

```
    self.last_response = ""
```

```
    self.last_error = ""
```

```
    # Inicia a thread de comunicação
```

```
    self.comm_thread.start()
```

```
def connect(self, port=None, baudrate=115200):
```

```
    """
```

```
    Conecta à máquina CNC.
```

```
    Args:
```

```
        port: Porta serial (None para autodetecção)
```

```
        baudrate: Taxa de transmissão
```

```
    Returns:
```

```
        bool: True se conectado com sucesso
```

```
    """
```

```
    try:
```

```
        # Autodetectar porta se não especificada
```

```
        if port is None:
```

```
            ports = [p.device for p in serial.tools.list_ports.comports()]
```

```
            if not ports:
```

```
                return False
```

```
            port = ports[0]
```

```
    serial_port = serial.Serial(port, baudrate, timeout=0.5)
```

```
    time.sleep(0.5)
```

```
    # Inicializar GRBL
```

```
    serial_port.write(b"\r\n\r\n")
```

```
    time.sleep(0.5)
```

```
    serial_port.flushInput()
```

```

# Configurar thread
self.comm_thread.set_serial_port(serial_port)

# Desbloquear e configurar
self.comm_thread.send_command("$X", priority=True)

self.is_connected = True
return True

except Exception as e:
    self.last_error = str(e)
    return False

def disconnect(self):
    """Desconecta da máquina CNC."""
    self.comm_thread.disconnect()
    self.is_connected = False

def is_moving(self):
    """Verifica se a máquina está em movimento."""
    return self.machine_status == "Run"

def wait_for_idle(self, timeout=30):
    """
    Aguarda até que a máquina esteja ociosa.

    Args:
        timeout: Tempo máximo de espera em segundos

    Returns:
        bool: True se ficou ocioso, False se atingiu timeout
    """
    start_time = time.time()
    while self.is_moving():
        if time.time() - start_time > timeout:
            return False
        time.sleep(0.1)
    return True

def get_current_position(self):
    """Retorna a posição atual da máquina."""
    return self.current_position.copy()

def move_to_absolute_position(self, x=None, y=None, feed_rate=1000):
    """
    Move para uma posição absoluta.

    Args:
        x, y: Coordenadas de destino (None para não mover esse eixo)
        feed_rate: Velocidade de avanço em mm/min
    """

```

```

"""
if not self.is_connected:
    return False

# Muda para modo absoluto
self.comm_thread.send_command("G90", priority=True)

# Constrói o comando
command = "G1"
if x is not None:
    command += f" X{x}"
if y is not None:
    command += f" Y{y}"
command += f" F{feed_rate}"

# Envia o comando
self.comm_thread.send_command(command)
return True

def move_relative(self, x=0, y=0, feed_rate=1000):
    """
    Move em relação à posição atual.

    Args:
        x, y: Distância a mover em cada eixo
        feed_rate: Velocidade de avanço em mm/min
    """
    if not self.is_connected:
        return False

    # Muda para modo relativo
    self.comm_thread.send_command("G91", priority=True)

    # Constrói o comando
    command = f"G1 X{x} Y{y} F{feed_rate}"

    # Envia o comando
    self.comm_thread.send_command(command)
    return True

def home(self):
    """Envia a máquina para a posição home."""
    if not self.is_connected:
        return False

    self.comm_thread.send_command("$H")
    return True

def set_zero(self):
    """Define a posição atual como zero."""
    if not self.is_connected:

```

```
return False
```

```
self.comm_thread.send_command("G92 X0 Y0")  
return True
```

```
def unlock(self):  
    """Desbloqueia a máquina."""  
    if not self.is_connected:  
        return False  
  
    self.comm_thread.send_command("$X", priority=True)  
    return True
```

```
def emergency_stop(self):  
    """Para todos os movimentos imediatamente."""  
    if not self.is_connected:  
        return False  
  
    self.comm_thread.send_command("!", priority=True)  
    return True
```

```
def send_command(self, command, priority=False):  
    """  
    Envia um comando personalizado ao GRBL.  
  
    Args:  
        command: Comando G-code  
        priority: True para enviar com prioridade  
    """  
    if not self.is_connected:  
        return False  
  
    self.comm_thread.send_command(command, priority)  
    return True
```

```
def _on_response_received(self, response):  
    """Callback quando uma resposta é recebida."""  
    self.last_response = response
```

```
def _on_status_update(self, status):  
    """Callback quando o status é atualizado."""  
    if status.startswith('<') and '|' in status:  
        self.machine_status = status[1:status.find('|')]
```

```
def _on_position_update(self, position):  
    """Callback quando a posição é atualizada."""  
    self.current_position = position
```

```
def _on_error_message(self, error):  
    """Callback quando ocorre um erro."""  
    self.last_error = error
```

File: position_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

import time

class InspectionPosition:

"""Representa uma posição de inspeção."""

def __init__(self, name, x, y, camera_params=None):

self.name = name

self.x = x

self.y = y

self.camera_params = camera_params or {}

self.creation_time = time.time()

def to_dict(self):

"""Converte para dicionário para serialização."""

return {

 'name': self.name,

 'x': self.x,

 'y': self.y,

 'camera_params': self.camera_params,

 'creation_time': self.creation_time

}

@classmethod

def from_dict(cls, data):

"""Cria a partir de um dicionário."""

position = cls(data['name'], data['x'], data['y'], data['camera_params'])

position.creation_time = data.get('creation_time', time.time())

return position

class InspectionSequence:

"""Representa uma sequência de posições de inspeção."""

def __init__(self, name):

self.name = name

self.positions = []

self.creation_time = time.time()

self.last_run_time = None

def add_position(self, position):

"""Adiciona uma posição à sequência."""

self.positions.append(position)

def remove_position(self, index):

"""Remove uma posição pelo índice."""

if 0 <= index < len(self.positions):

del self.positions[index]

return True

```
return False
```

```
def move_position(self, old_index, new_index):  
    """Move uma posição na sequência."""  
    if 0 <= old_index < len(self.positions) and 0 <= new_index < len(self.positions):  
        position = self.positions.pop(old_index)  
        self.positions.insert(new_index, position)  
        return True  
    return False
```

```
def to_dict(self):  
    """Converte para dicionário para serialização."""  
    return {  
        'name': self.name,  
        'positions': [p.to_dict() for p in self.positions],  
        'creation_time': self.creation_time,  
        'last_run_time': self.last_run_time  
    }
```

```
@classmethod
```

```
def from_dict(cls, data, positions_dict=None):  
    """Cria a partir de um dicionário."""  
    sequence = cls(data['name'])  
    sequence.creation_time = data.get('creation_time', time.time())  
    sequence.last_run_time = data.get('last_run_time')  
  
    # Reconstruir posições  
    for pos_data in data['positions']:  
        if positions_dict and pos_data['name'] in positions_dict:  
            # Usa posição existente  
            sequence.positions.append(positions_dict[pos_data['name']])  
        else:  
            # Cria nova posição  
            position = InspectionPosition.from_dict(pos_data)  
            sequence.positions.append(position)  
  
    return sequence
```

```
class InspectionPositionManager:  
    """Gerencia posições e sequências de inspeção."""  
  
    def __init__(self):  
        self.positions = {} # name -> position  
        self.sequences = {} # name -> sequence  
  
    def add_position(self, position):  
        """Adiciona uma posição."""  
        self.positions[position.name] = position  
  
    def remove_position(self, position_name):
```

```

"""Remove uma posição pelo nome."""
if position_name in self.positions:
    del self.positions[position_name]

    # Atualiza sequências que usam esta posição
    for sequence in self.sequences.values():
        sequence.positions = [p for p in sequence.positions if p.name != position_name]

    return True
return False

def get_position(self, position_name):
    """Obtém uma posição pelo nome."""
    return self.positions.get(position_name)

def add_sequence(self, sequence):
    """Adiciona uma sequência."""
    self.sequences[sequence.name] = sequence

def remove_sequence(self, sequence_name):
    """Remove uma sequência pelo nome."""
    if sequence_name in self.sequences:
        del self.sequences[sequence_name]
        return True
    return False

def get_sequence(self, sequence_name):
    """Obtém uma sequência pelo nome."""
    return self.sequences.get(sequence_name)

def save_to_file(self, filename):
    """Salva todas as posições e sequências em um arquivo JSON."""
    data = {
        'positions': {name: pos.to_dict() for name, pos in self.positions.items()},
        'sequences': {name: seq.to_dict() for name, seq in self.sequences.items()}
    }

    try:
        with open(filename, 'w') as f:
            import json
            json.dump(data, f, indent=2)
        return True
    except Exception as e:
        print(f"Erro ao salvar dados: {e}")
        return False

def load_from_file(self, filename):
    """Carrega posições e sequências de um arquivo JSON."""
    try:
        with open(filename, 'r') as f:
            import json

```

```

        data = json.load(f)

        # Limpa dados existentes
        self.positions.clear()
        self.sequences.clear()

        # Carrega posições
        for name, pos_data in data.get('positions', {}).items():
            position = InspectionPosition.from_dict(pos_data)
            self.positions[name] = position

        # Carrega sequências
        for name, seq_data in data.get('sequences', {}).items():
            sequence = InspectionSequence.from_dict(seq_data, self.positions)
            self.sequences[name] = sequence

        return True

    except Exception as e:
        print(f"Erro ao carregar dados: {e}")
        return False

```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib

```

# Exporta as classes principais para facilitar importação
from .aoi_controller import CNCAOIController
from .cnc_controller import GRBLCNCCController
from .camera_controller import CameraController
from .position_manager import InspectionPosition, InspectionSequence

__version__ = '0.1.0'

```

File: config_manager.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib\utils

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi/aoi_lib\utils

File: consumo_lib.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi

```

import sys
import cv2

```

```

import time
import numpy as np
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
                             QPushButton, QLabel, QGroupBox, QGridLayout, QLineEdit,
                             QComboBox, QListWidget, QListWidgetItem, QFileDialog, QMessageBox, QTabWidget,
                             QSplitter, QFrame, QTableWidget, QTableWidgetItem, QHeaderView)
from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer
from PyQt6.QtGui import QPixmap, QImage, QFont

from aoi_lib import CNCAOIController, InspectionPosition

class ImageViewerWidget(QWidget):
    """Widget para exibir imagens capturadas pela câmera"""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.layout = QVBoxLayout(self)

        # Label para exibir a imagem
        self.image_label = QLabel()
        self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.image_label.setText("Nenhuma imagem capturada")
        self.image_label.setStyleSheet("border: 1px solid gray; background-color: #f0f0f0;")
        self.image_label.setMinimumSize(400, 300)

        # Informações da imagem
        self.info_label = QLabel("Informações da imagem:")

        self.layout.addWidget(self.image_label)
        self.layout.addWidget(self.info_label)

    def display_image(self, image, info_text=None):
        """Mostra uma imagem no widget"""
        if image is None:
            self.image_label.setText("Imagem inválida")
            return

        # Converte imagem OpenCV para QPixmap
        h, w, c = image.shape
        bytes_per_line = 3 * w
        q_img = QImage(image.data, w, h, bytes_per_line, QImage.Format.Format_RGB888).rgbSwapped()
        pixmap = QPixmap.fromImage(q_img)

        # Redimensiona se for muito grande
        if pixmap.width() > 800 or pixmap.height() > 600:
            pixmap = pixmap.scaled(800, 600, Qt.AspectRatioMode.KeepAspectRatio)

        self.image_label.setPixmap(pixmap)

        # Atualiza informações
        if info_text:
            self.info_label.setText(info_text)

```

else:

self.info_label.setText(f"Imagem: {w}x{h}px")

class PositionListWidget(QWidget):

"""Widget para gerenciar lista de posições"""

position_selected = pyqtSignal(object) # Emite a posição selecionada

def __init__(self, parent=None):

super().__init__(parent)

self.layout = QVBoxLayout(self)

Título

title_label = QLabel("Posições de Inspeção")

title_font = QFont()

title_font.setBold(True)

title_label.setFont(title_font)

Lista de posições

self.positions_list = QListWidget()

self.positions_list.currentItemChanged.connect(self.on_position_selected)

Botões

buttons_layout = QHBoxLayout()

self.add_position_btn = QPushButton("Adicionar Posição Atual")

self.remove_position_btn = QPushButton("Remover")

buttons_layout.addWidget(self.add_position_btn)

buttons_layout.addWidget(self.remove_position_btn)

self.layout.addWidget(title_label)

self.layout.addWidget(self.positions_list)

self.layout.addLayout(buttons_layout)

Dicionário para mapear itens da lista para objetos de posição

self.positions = {}

def add_position(self, position):

"""Adiciona uma posição à lista"""

item_text = f"{{position.name}} ({{position.x:.2f}}, {{position.y:.2f}})"

item = QListWidgetItem(item_text)

self.positions_list.addItem(item)

self.positions[item] = position

def remove_selected_position(self):

"""Remove a posição selecionada"""

current_item = self.positions_list.currentItem()

if current_item:

position = self.positions.pop(current_item)

row = self.positions_list.row(current_item)

self.positions_list.takeItem(row)

return position

return None

```

def clear_positions(self):
    """Limpa todas as posições"""
    self.positions_list.clear()
    self.positions = {}

def get_all_positions(self):
    """Retorna todas as posições"""
    return list(self.positions.values())

def on_position_selected(self, current, previous):
    """Manipula evento de seleção de posição"""
    if current and current in self.positions:
        self.position_selected.emit(self.positions[current])

class SequenceControlWidget(QWidget):
    """Widget para controlar a execução da sequência"""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.layout = QVBoxLayout(self)

        # Grupo de sequência
        sequence_group = QGroupBox("Controle de Sequência")
        sequence_layout = QGridLayout()

        # Nome da sequência
        sequence_layout.addWidget(QLabel("Nome:"), 0, 0)
        self.sequence_name = QLineEdit("Sequência PCB")
        sequence_layout.addWidget(self.sequence_name, 0, 1)

        # Botões de controle
        self.create_sequence_btn = QPushButton("Criar Sequência")
        self.run_sequence_btn = QPushButton("Executar Sequência")
        self.stop_sequence_btn = QPushButton("Parar")
        self.stop_sequence_btn.setEnabled(False)

        sequence_layout.addWidget(self.create_sequence_btn, 1, 0)
        sequence_layout.addWidget(self.run_sequence_btn, 1, 1)
        sequence_layout.addWidget(self.stop_sequence_btn, 2, 0, 1, 2)

        # Status
        sequence_layout.addWidget(QLabel("Status:"), 3, 0)
        self.sequence_status = QLabel("Pronto")
        sequence_layout.addWidget(self.sequence_status, 3, 1)

        sequence_group.setLayout(sequence_layout)

        # Grupo de arquivo
        file_group = QGroupBox("Salvar/Carregar")
        file_layout = QVBoxLayout()

```

```
self.save_btn = QPushButton("Salvar Programa")
self.load_btn = QPushButton("Carregar Programa")
```

```
file_layout.addWidget(self.save_btn)
file_layout.addWidget(self.load_btn)
```

```
file_group.setLayout(file_layout)
```

```
self.layout.addWidget(sequence_group)
self.layout.addWidget(file_group)
self.layout.addStretch()
```

```
class AOIControllerApp(QMainWindow):
```

```
    """Aplicação principal para controle do sistema AOI"""
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.setWindowTitle("Controle de Inspeção Óptica Automatizada")
```

```
        self.setGeometry(100, 100, 1200, 800)
```

```
        # Inicializa o controlador AOI
```

```
        self.controller = CNCAOIController()
```

```
        self.current_sequence = None
```

```
        self.is_running_sequence = False
```

```
        # Configuração da interface
```

```
        self.setup_ui()
```

```
        # Timer para atualizar a posição
```

```
        self.update_timer = QTimer(self)
```

```
        self.update_timer.timeout.connect(self.update_position_display)
```

```
        self.update_timer.start(500) # Atualiza a cada 500ms
```

```
    def setup_ui(self):
```

```
        # Widget central
```

```
        central_widget = QWidget()
```

```
        self.setCentralWidget(central_widget)
```

```
        # Layout principal
```

```
        main_layout = QVBoxLayout(central_widget)
```

```
        # Grupo de conexão
```

```
        connection_group = QGroupBox("Conexão")
```

```
        connection_layout = QGridLayout()
```

```
        # CNC Connection
```

```
        connection_layout.addWidget(QLabel("Porta CNC:"), 0, 0)
```

```
        self.cnc_port_combo = QComboBox()
```

```
        self.refresh_ports()
```

```
        connection_layout.addWidget(self.cnc_port_combo, 0, 1)
```

```
        self.connect_cnc_btn = QPushButton("Conectar CNC")
```

```

self.connect_cnc_btn.clicked.connect(self.connect_cnc)
connection_layout.addWidget(self.connect_cnc_btn, 0, 2)

# Camera connection
connection_layout.addWidget(QLabel("Câmera ID:"), 1, 0)
self.camera_id_combo = QComboBox()
self.camera_id_combo.addItems(["0", "1", "2", "3"])
connection_layout.addWidget(self.camera_id_combo, 1, 1)

self.connect_camera_btn = QPushButton("Conectar Câmera")
self.connect_camera_btn.clicked.connect(self.connect_camera)
connection_layout.addWidget(self.connect_camera_btn, 1, 2)

# Refresh ports button
self.refresh_ports_btn = QPushButton("Atualizar Portas")
self.refresh_ports_btn.clicked.connect(self.refresh_ports)
connection_layout.addWidget(self.refresh_ports_btn, 0, 3)

# Test camera button
self.test_camera_btn = QPushButton("Testar Câmera")
self.test_camera_btn.clicked.connect(self.test_camera)
connection_layout.addWidget(self.test_camera_btn, 1, 3)

connection_group.setLayout(connection_layout)
main_layout.addWidget(connection_group)

# Splitter para dividir a interface em painéis
splitter = QSplitter(Qt.Orientation.Horizontal)

# Painel esquerdo: controles e lista de posições
left_panel = QWidget()
left_layout = QVBoxLayout(left_panel)

# Informações de posição
position_group = QGroupBox("Posição Atual")
position_layout = QGridLayout()

position_layout.addWidget(QLabel("X:"), 0, 0)
self.x_position = QLabel("0.000 mm")
position_layout.addWidget(self.x_position, 0, 1)

position_layout.addWidget(QLabel("Y:"), 1, 0)
self.y_position = QLabel("0.000 mm")
position_layout.addWidget(self.y_position, 1, 1)

position_layout.addWidget(QLabel("Status:"), 2, 0)
self.cnc_status = QLabel("Desconectado")
position_layout.addWidget(self.cnc_status, 2, 1)

position_group.setLayout(position_layout)
left_layout.addWidget(position_group)

```

```

# Widget de lista de posições
self.position_list_widget = PositionListWidget()
self.position_list_widget.add_position_btn.clicked.connect(self.add_current_position)
self.position_list_widget.remove_position_btn.clicked.connect(self.remove_position)
self.position_list_widget.position_selected.connect(self.on_position_selected)

left_layout.addWidget(self.position_list_widget)

# Controle de sequência
self.sequence_widget = SequenceControlWidget()
self.sequence_widget.create_sequence_btn.clicked.connect(self.create_sequence)
self.sequence_widget.run_sequence_btn.clicked.connect(self.run_sequence)
self.sequence_widget.stop_sequence_btn.clicked.connect(self.stop_sequence)
self.sequence_widget.save_btn.clicked.connect(self.save_program)
self.sequence_widget.load_btn.clicked.connect(self.load_program)

left_layout.addWidget(self.sequence_widget)

# Painel direito: visualizador de imagem e resultados
right_panel = QTabWidget()

# Aba de visualização de imagem
self.image_viewer = ImageViewerWidget()
right_panel.addTab(self.image_viewer, "Visualizador de Imagem")

# Aba de resultados
results_widget = QWidget()
results_layout = QVBoxLayout(results_widget)

self.results_table = QTableWidgetItem(0, 3)
self.results_table.setHorizontalHeaderLabels(["Posição", "Timestamp", "Status"])
self.results_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)

results_layout.addWidget(QLabel("Resultados da Inspeção:"))
results_layout.addWidget(self.results_table)

right_panel.addTab(results_widget, "Resultados")

# Adiciona painéis ao splitter
splitter.addWidget(left_panel)
splitter.addWidget(right_panel)
splitter.setSizes([400, 800])

main_layout.addWidget(splitter)

# Barra de status
self.statusBar().showMessage("Pronto para conectar")

def refresh_ports(self):
    """Atualiza a lista de portas seriais disponíveis"""

```

```

import serial.tools.list_ports

self.cnc_port_combo.clear()
ports = [port.device for port in serial.tools.list_ports.comports()]

if ports:
    self.cnc_port_combo.addItems(ports)

    # Verificar se COM9 está na lista e selecionar
    com9_index = self.cnc_port_combo.findText("COM9")
    if com9_index >= 0:
        self.cnc_port_combo.setCurrentIndex(com9_index)
        self.statusBar().showMessage("Porta COM9 detectada")
    else:
        self.statusBar().showMessage("Nenhuma porta serial encontrada")

def connect_cnc(self):
    """Conecta à máquina CNC"""
    if self.controller.cnc.is_connected:
        # Desconectar
        self.controller.cnc.disconnect()
        self.connect_cnc_btn.setText("Conectar CNC")
        self.cnc_status.setText("Desconectado")
        self.statusBar().showMessage("CNC desconectada")
    else:
        # Conectar
        port = self.cnc_port_combo.currentText()
        if not port:
            QMessageBox.warning(self, "Erro", "Selecione uma porta serial")
            return

        self.statusBar().showMessage(f"Conectando à CNC na porta {port}...")

        if self.controller.connect_cnc(port):
            self.connect_cnc_btn.setText("Desconectar CNC")
            self.cnc_status.setText("Conectado")
            self.statusBar().showMessage(f"CNC conectada na porta {port}")
        else:
            QMessageBox.critical(self, "Erro", f"Falha ao conectar à CNC: {self.controller.cnc.last_error}")

def connect_camera(self):
    """Conecta à câmera"""
    if hasattr(self.controller.camera, 'is_connected') and self.controller.camera.is_connected:
        # Desconectar
        self.controller.camera.disconnect()
        self.connect_camera_btn.setText("Conectar Câmera")
        self.statusBar().showMessage("Câmera desconectada")
    else:
        # Conectar
        try:
            camera_id = int(self.camera_id_combo.currentText())

```

```

self.statusBar().showMessage(f"Conectando à câmera ID {camera_id}...")

if self.controller.connect_camera(camera_id):
    self.connect_camera_btn.setText("Desconectar Câmera")
    self.statusBar().showMessage(f"Câmera ID {camera_id} conectada")
else:
    QMessageBox.critical(self, "Erro", f"Falha ao conectar à câmera: {self.controller.camera.last_error}")
except ValueError:
    QMessageBox.warning(self, "Erro", "ID de câmera inválido")

def test_camera(self):
    """Testa a captura de imagem da câmera"""
    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
        QMessageBox.warning(self, "Aviso", "Câmera não conectada")
    return

image = self.controller.camera.capture()

if image is not None:
    self.image_viewer.display_image(image, "Imagem de teste da câmera")
    self.statusBar().showMessage("Imagem capturada com sucesso")
else:
    QMessageBox.warning(self, "Erro", f"Falha ao capturar imagem: {self.controller.camera.last_error}")

def update_position_display(self):
    """Atualiza a exibição da posição atual"""
    if not self.controller.cnc.is_connected:
        return

    position = self.controller.cnc.get_current_position()
    self.x_position.setText(f"{position['x']:.3f} mm")
    self.y_position.setText(f"{position['y']:.3f} mm")

    # Atualiza status
    self.cnc_status.setText(self.controller.cnc.machine_status)

def add_current_position(self):
    """Adiciona a posição atual à lista"""
    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Aviso", "CNC não conectada")
    return

position_name, ok = QInputDialog.getText(self, "Nova Posição", "Nome da posição:")

if ok and position_name:
    position = self.controller.add_current_position(position_name)
    self.position_list_widget.add_position(position)
    self.statusBar().showMessage(f"Posição '{position_name}' adicionada")

def remove_position(self):

```

```

"""Remove a posição selecionada"""
position = self.position_list_widget.remove_selected_position()

if position:
    self.controller.position_manager.remove_position(position.name)
    self.statusBar().showMessage(f"Posição '{position.name}' removida")

def on_position_selected(self, position):
    """Manipula a seleção de uma posição"""
    # Poderia mover para esta posição, mostrar detalhes, etc.
    self.statusBar().showMessage(f"Posição selecionada: {position.name} ({position.x:.3f}, {position.y:.3f})")

def create_sequence(self):
    """Cria uma nova sequência com as posições atuais"""
    positions = self.position_list_widget.get_all_positions()

    if not positions:
        QMessageBox.warning(self, "Aviso", "Adicione pelo menos uma posição")
        return

    sequence_name = self.sequence_widget.sequence_name.text()
    if not sequence_name:
        QMessageBox.warning(self, "Aviso", "Digite um nome para a sequência")
        return

    self.current_sequence = self.controller.create_sequence(sequence_name, positions)
    self.statusBar().showMessage(f"Sequência '{sequence_name}' criada com {len(positions)} posições")
    self.sequence_widget.sequence_status.setText(f"Criada: {len(positions)} posições")

def run_sequence(self):
    """Executa a sequência atual"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
        return

    if not self.controller.cnc.is_connected:
        QMessageBox.warning(self, "Aviso", "CNC não conectada")
        return

    if not hasattr(self.controller.camera, 'is_connected') or not self.controller.camera.is_connected:
        QMessageBox.warning(self, "Aviso", "Câmera não conectada")
        return

    # Limpa resultados anteriores
    self.results_table.setRowCount(0)

    # Configura UI para execução
    self.is_running_sequence = True
    self.sequence_widget.run_sequence_btn.setEnabled(False)
    self.sequence_widget.stop_sequence_btn.setEnabled(True)
    self.sequence_widget.sequence_status.setText("Executando...")

```

```

# Inicia a execução
try:
    self.statusBar().showMessage(f"Executando sequência '{self.current_sequence.name}'...")

    # Nesta versão de GUI, usaremos um thread separado para executar a sequência
    self.run_thread = SequenceRunnerThread(self.controller, self.current_sequence.name)
    self.run_thread.image_captured.connect(self.on_image_captured)
    self.run_thread.sequence_completed.connect(self.on_sequence_completed)
    self.run_thread.sequence_error.connect(self.on_sequence_error)
    self.run_thread.start()

except Exception as e:
    self.statusBar().showMessage(f"Erro ao executar sequência: {str(e)}")
    self.sequence_widget.sequence_status.setText("Erro")
    self.on_sequence_completed()

def stop_sequence(self):
    """Para a execução da sequência atual"""
    if self.is_running_sequence:
        self.controller.stop_sequence()
        self.statusBar().showMessage("Execução de sequência interrompida")
        self.on_sequence_completed()

def on_image_captured(self, result):
    """Chamado quando uma imagem é capturada durante a execução da sequência"""
    position = result["position"]
    image = result["image"]
    timestamp = result["timestamp"]

    # Exibe a imagem
    self.image_viewer.display_image(image, f"Posição: {position.name} ({position.x:.3f}, {position.y:.3f})")

    # Adiciona ao registro de resultados
    row = self.results_table.rowCount()
    self.results_table.insertRow(row)
    self.results_table.setItem(row, 0, QTableWidgetItem(position.name))
    self.results_table.setItem(row, 1, QTableWidgetItem(time.strftime("%H:%M:%S", time.localtime(timestamp))))
    self.results_table.setItem(row, 2, QTableWidgetItem("Capturado"))

def on_sequence_completed(self):
    """Chamado quando a execução da sequência é concluída"""
    self.is_running_sequence = False
    self.sequence_widget.run_sequence_btn.setEnabled(True)
    self.sequence_widget.stop_sequence_btn.setEnabled(False)
    self.sequence_widget.sequence_status.setText("Concluída")
    self.statusBar().showMessage("Execução de sequência concluída")

def on_sequence_error(self, error_message):
    """Chamado quando ocorre um erro durante a execução da sequência"""
    QMessageBox.critical(self, "Erro na Sequência", error_message)

```

```

self.on_sequence_completed()

def save_program(self):
    """Salva o programa de inspeção atual"""
    if not self.current_sequence:
        QMessageBox.warning(self, "Aviso", "Crie uma sequência primeiro")
        return

    filename, _ = QFileDialog.getSaveFileName(self, "Salvar Programa", "", "Arquivos JSON (*.json)")

    if filename:
        if not filename.endswith('.json'):
            filename += '.json'

        if self.controller.save_positions(filename):
            self.statusBar().showMessage(f"Programa salvo em {filename}")
        else:
            QMessageBox.critical(self, "Erro", "Falha ao salvar o programa")

def load_program(self):
    """Carrega um programa de inspeção salvo"""
    filename, _ = QFileDialog.getOpenFileName(self, "Carregar Programa", "", "Arquivos JSON (*.json)")

    if filename:
        if self.controller.load_positions(filename):
            # Atualiza a interface
            self.position_list_widget.clear_positions()

            # Adiciona posições carregadas à lista
            for name, position in self.controller.position_manager.positions.items():
                self.position_list_widget.add_position(position)

            # Se há sequências, usa a primeira como atual
            if self.controller.position_manager.sequences:
                sequence_name = next(iter(self.controller.position_manager.sequences))
                self.current_sequence = self.controller.position_manager.sequences[sequence_name]
                self.sequence_widget.sequence_name.setText(sequence_name)
                self.sequence_widget.sequence_status.setText(f"Carregada: {len(self.current_sequence.positions)} posições")

            self.statusBar().showMessage(f"Programa carregado de {filename}")
        else:
            QMessageBox.critical(self, "Erro", "Falha ao carregar o programa")

def closeEvent(self, event):
    """Manipula o evento de fechamento da janela"""
    # Para a execução da sequência, se houver
    if self.is_running_sequence:
        self.controller.stop_sequence()

    # Desconecta da câmera e CNC
    if hasattr(self.controller.camera, 'is_connected') and self.controller.camera.is_connected:

```

```

        self.controller.camera.disconnect()

    if self.controller.cnc.is_connected:
        self.controller.cnc.disconnect()

    event.accept()

class SequenceRunnerThread(QThread):
    """Thread para executar uma sequência de inspeção"""
    image_captured = pyqtSignal(dict) # Emite resultados da captura
    sequence_completed = pyqtSignal() # Emite quando a sequência é concluída
    sequence_error = pyqtSignal(str) # Emite quando ocorre um erro

    def __init__(self, controller, sequence_name):
        super().__init__()
        self.controller = controller
        self.sequence_name = sequence_name

    def run(self):
        try:
            # Execute a sequência e processe os resultados
            def process_result(result):
                self.image_captured.emit(result)

            self.controller.run_sequence(self.sequence_name, process_result)
            self.sequence_completed.emit()

        except Exception as e:
            self.sequence_error.emit(str(e))

# Classe QDialog que estava faltando
from PyQt6.QtWidgets import QDialog

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = AOIControllerApp()
    window.show()
    sys.exit(app.exec())

```

File: controlador.py

Directory: C:/Users/sense/PycharmProjects/cnc_aoi

```

import sys
import serial
import serial.tools.list_ports
import time
import re
import threading
import logging
from queue import Queue, Empty, PriorityQueue

```

```

from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QPushButton, QLabel, QSlider, QGroupBox,
                             QGridLayout, QLineEdit, QComboBox, QProgressBar,
                             QFrame, QPushButton)
from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer
from PyQt6.QtGui import QFont, QColor, QPalette, QIcon

```

```

# Configuração de logging para diagnóstico

```

```

logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("grbl_comm.log"),
        logging.StreamHandler()
    ]
)

```

```

logger = logging.getLogger("GRBLComm")

```

```

class GRBLCommunicationThread(QThread):

```

```

    """Thread para lidar com a comunicação com GRBL sem bloquear a interface"""

```

```

    response_received = pyqtSignal(str)

```

```

    status_update = pyqtSignal(str)

```

```

    position_update = pyqtSignal(dict)

```

```

    error_message = pyqtSignal(str)

```

```

    def __init__(self, parent=None):

```

```

        super().__init__(parent)

```

```

        self.serial_port = None

```

```

        self.is_connected = False

```

```

        self.running = True

```

```

        # Usando PriorityQueue como em consumo_lib.py

```

```

        self.command_queue = PriorityQueue()

```

```

        # Usando um único lock para simplicidade

```

```

        self.lock = threading.Lock()

```

```

        self.is_moving = False

```

```

        self.last_status_check = 0

```

```

        self.last_position = {'x': 0, 'y': 0, 'z': 0}

```

```

        self.machine_state = "Unknown"

```

```

        logger.info("GRBLCommunicationThread inicializada")

```

```

    def set_serial_port(self, serial_port):

```

```

        """Define a porta serial para comunicação."""

```

```

        logger.info(f"Definindo porta serial: {serial_port}")

```

```

        self.serial_port = serial_port

```

```

        self.is_connected = True

```

```

    def disconnect(self):

```

```

        """Desconecta a porta serial."""

```

```

        logger.info("Desconectando porta serial")

```

```

self.is_connected = False
if self.serial_port and self.serial_port.is_open:
    try:
        with self.lock:
            logger.debug("Fechando porta serial")
            self.serial_port.close()
            logger.info("Porta serial fechada com sucesso")
    except Exception as e:
        logger.error(f"Erro ao fechar porta serial: {e}")

```

```

def send_command(self, command, priority=False):

```

```

    """

```

Adiciona um comando à fila de execução, com opção de prioridade.

Args:

command: O comando GCODE

priority: Se True, o comando tem prioridade

```

    """

```

Comandos de controle imediato (!, ~, ?) são sempre processados imediatamente

```

if command in ['!', '~', '?']:

```

```

    priority_level = -1 # Prioridade máxima

```

```

else:

```

```

    priority_level = 0 if priority else 1

```

```

logger.debug(f"Enviando comando: '{command}' (priority={priority_level})")

```

```

self.command_queue.put((priority_level, command))

```

```

def run(self):

```

```

    """Método principal da thread - processa os comandos e monitora o status"""

```

```

logger.info("Thread de comunicação iniciada")

```

```

last_diagnostic_time = 0

```

```

last_command_time = 0

```

```

while self.running:

```

```

    try:

```

```

        # Diagnóstico periódico

```

```

        current_time = time.time()

```

```

        if current_time - last_diagnostic_time > 10:

```

```

            if self.is_connected and self.serial_port and self.serial_port.is_open:

```

```

                buffer_size = self.serial_port.in_waiting if hasattr(self.serial_port, 'in_waiting') else 0

```

```

                logger.info(f"Diagnóstico de comunicação: Conectado, buffer_in={buffer_size}, is_moving={self.is_moving}, e

```

```

            # Limpar buffer se estiver muito grande

```

```

            if buffer_size > 200:

```

```

                logger.warning(f"Buffer muito grande ({buffer_size} bytes), limpando...")

```

```

                with self.lock:

```

```

                    self.serial_port.reset_input_buffer()

```

```

            else:

```

```

                logger.info(f"Diagnóstico de comunicação: Desconectado, is_moving={self.is_moving}, estado={self.machine

```

```

last_diagnostic_time = current_time

```

```

# Verificar conexão
if not self.is_connected or not self.serial_port or not self.serial_port.is_open:
    logger.debug("Não conectado, aguardando...")
    time.sleep(0.1)
    continue

# Verificar e processar dados recebidos mesmo sem enviar comando
if self.serial_port.in_waiting > 0:
    with self.lock:
        data = self.serial_port.readline().decode('utf-8', errors='replace').strip()

    if data:
        logger.debug(f"Dados recebidos: '{data}'")

        # Processar dados recebidos
        if data.startswith('<') and '|' in data:
            self._parse_status(data)
        elif "ok" in data or "error" in data:
            self.response_received.emit(data)

# Limita a taxa de envio de comandos
can_send_command = True
if current_time - last_command_time < 0.1: # Máximo 10 comandos por segundo
    can_send_command = False

# Processa comandos na fila
if can_send_command and not self.command_queue.empty():
    priority, command = self.command_queue.get(block=False)

    # Comandos prioritários ou de controle podem sempre ser enviados
    if priority <= 0 or command in ['!', '~', '?']:
        pass # Sempre permite
    # Comandos regulares verificam o estado da máquina
    elif self.is_moving and "Hold" not in self.machine_state:
        logger.debug(f"Máquina em movimento, enfileirando comando: {command}")
        self.command_queue.put((priority, command))
        self.command_queue.task_done()
        time.sleep(0.05)
        continue

# Preparar comando
if not command.endswith('\n'):
    command += '\n'

# Identificar tipo de comando
is_movement_cmd = any(cmd in command for cmd in ['G0', 'G1', 'G2', 'G3'])

# Enviar comando
with self.lock:
    logger.debug(f"Enviando bytes: {command.encode()}")
    self.serial_port.write(command.encode())

```

```

self.serial_port.flush()

# Se for comando de movimento, atualiza estado
if is_movement_cmd:
    logger.info("Atualizando flag is_moving para True")
    self.is_moving = True

# Atualiza timestamp
last_command_time = current_time

# Para comandos rápidos (controle), não espera resposta
if command.strip() in ['!', '~', '?']:
    self.command_queue.task_done()
    continue

# Para outros comandos, espera resposta
start_time = time.time()
response = ""

# Lê respostas com timeout curto (200ms)
while (time.time() - start_time) < 0.2:
    if self.serial_port.in_waiting > 0:
        line = self.serial_port.readline().decode('utf-8', errors='replace').strip()
        if line:
            if not line.startswith('<'):
                response += line + " "
            # Se recebeu ok ou error, finaliza
            if "ok" in line or "error" in line:
                break
            # Status atualiza estado mas não é considerado resposta final
            elif line.startswith('<') and '|' in line:
                self._parse_status(line)
        else:
            time.sleep(0.01)

# Finaliza comando
self.command_queue.task_done()

# Processa resposta
if response:
    self.response_received.emit(response.strip())
    if is_movement_cmd and "error" in response.lower():
        logger.warning(f"Erro GRBL em comando de movimento: {response}")
        self.error_message.emit(f"Erro GRBL: {response}")
        self.is_moving = False

# Verificação periódica de status
if (current_time - self.last_status_check) > 0.2:
    self._check_status_simple()
    self.last_status_check = current_time

```

```
# Pausa pequena para não consumir CPU
time.sleep(0.01)
```

```
except Exception as e:
```

```
    logger.error(f"Erro na thread de comunicação: {e}", exc_info=True)
    self.error_message.emit(f"Erro na thread de comunicação: {str(e)}")
    time.sleep(0.1)
```

```
def _check_status_simple(self):
```

```
    """Verifica o status atual do GRBL (implementação simplificada)"""
```

```
    try:
```

```
        if not self.is_connected or not self.serial_port or not self.serial_port.is_open:
            return
```

```
        # Apenas envia o comando de status
```

```
        with self.lock:
```

```
            logger.debug("Enviando comando de status '?'")
```

```
            self.serial_port.write(b"?.")
```

```
            self.serial_port.flush()
```

```
            # Não aguardamos resposta aqui, ela será processada no próximo ciclo
```

```
    except Exception as e:
```

```
        logger.error(f"Erro ao verificar status: {e}")
```

```
# Modifique o método _parse_status na classe GRBLCommunicationThread
```

```
def _parse_status(self, status_line):
```

```
    """
```

```
    Analisa a linha de status do GRBL.
```

```
    Formato típico: <Idle|MPos:0.000,0.000,0.000|FS:0,0|WCO:0.000,0.000,0.000>
```

```
    """
```

```
    try:
```

```
        logger.debug(f"Analisando status: '{status_line}'")
```

```
        # Extrai o estado da máquina
```

```
        if status_line.startswith('<') and '|' in status_line:
```

```
            state = status_line[1:status_line.find('|')]
```

```
            old_state = self.machine_state
```

```
            self.machine_state = state
```

```
            logger.debug(f"Estado da máquina: {old_state} -> {state}")
```

```
        # Atualiza a flag de movimento com base no estado
```

```
        old_moving = self.is_moving
```

```
        # Estados onde a máquina está definitivamente parada
```

```
        if state in ["Idle", "Alarm", "Error", "Door", "Check", "Home"]:
```

```
            self.is_moving = False
```

```
        # Estados onde a máquina está definitivamente em movimento
```

```
        elif state in ["Run", "Jog"]:
```

```
            self.is_moving = True
```

```
        # Estado Hold - a máquina está temporariamente parada, mas não completamente Idle
```

```

elif state.startswith("Hold"):
    # Para os nossos propósitos, consideramos Hold como "não está em movimento"
    # Isso permite que novos comandos sejam enviados após liberar o Hold
    self.is_moving = False

if old_moving != self.is_moving:
    logger.info(f"Flag is_moving atualizada: {old_moving} -> {self.is_moving}")

# Emite estado para atualizar UI quando o estado muda
if old_state != state:
    self.status_update.emit(status_line)

# Extrai a posição
match = re.search(r'(?MPos|WPos):(-?\d+\.\d+),(-?\d+\.\d+),(-?\d+\.\d+)', status_line)
if match:
    x, y, z = match.groups()
    position = {
        'x': float(x),
        'y': float(y),
        'z': float(z)
    }

    # Emite atualização apenas se a posição mudou significativamente (> 0.001mm)
    if (abs(position['x'] - self.last_position['x']) > 0.001 or
        abs(position['y'] - self.last_position['y']) > 0.001 or
        abs(position['z'] - self.last_position['z']) > 0.001):

        self.position_update.emit(position)
        self.last_position = position

except Exception as e:
    logger.error(f"Erro ao analisar status: {e}", exc_info=True)

```

```

class DirectionalButton(QPushButton):
    """Botão direcional com setas para controle intuitivo de movimento"""
    def __init__(self, direction, parent=None):
        super().__init__(parent)
        self.direction = direction

    # Configura a aparência do botão
    self.setMinimumSize(60, 60)
    self.setMaximumSize(60, 60)

    # Define texto/símbolo com base na direção
    if direction == "up":
        self.setText("↑")
    elif direction == "down":
        self.setText("↓")
    elif direction == "left":
        self.setText("←")
    elif direction == "right":

```

```
self.setText("→")
```

```
# Estiliza o botão para parecer com botão de jog
```

```
font = QFont()
```

```
font.setBold(True)
```

```
font.setPointSize(20)
```

```
self.setFont(font)
```

```
# Configuração para repetição automática - ajustada para ser mais lenta e estável
```

```
self.setAutoRepeat(True)
```

```
self.setAutoRepeatDelay(500) # Aumentado para 500ms - espera mais antes de começar a repetir
```

```
self.setAutoRepeatInterval(150) # Aumentado para 150ms - repete mais lentamente
```

```
class JogButton(QPushButton):
```

```
    """Botão personalizado para movimentos jog"""
```

```
    def __init__(self, text, parent=None):
```

```
        super().__init__(text, parent)
```

```
        self.setAutoRepeat(True)
```

```
        self.setAutoRepeatDelay(500) # Aumentado para 500ms
```

```
        self.setAutoRepeatInterval(150) # Aumentado para 150ms
```

```
    # Estilização
```

```
    self.setMinimumSize(60, 60)
```

```
    font = QFont()
```

```
    font.setBold(True)
```

```
    font.setPointSize(12)
```

```
    self.setFont(font)
```

```
class PositionDisplay(QFrame):
```

```
    """Widget para exibir a posição atual da máquina"""
```

```
    def __init__(self, parent=None):
```

```
        super().__init__(parent)
```

```
        self setFrameShape(QFrame.Shape.StyledPanel)
```

```
        self setFrameShadow(QFrame.Shadow.Raised)
```

```
        layout = QGridLayout(self)
```

```
    # Rótulos para cabeçalhos
```

```
    title_font = QFont()
```

```
    title_font.setBold(True)
```

```
    title_label = QLabel("Posição Atual:")
```

```
    title_label.setFont(title_font)
```

```
    layout.addWidget(title_label, 0, 0, 1, 2)
```

```
    layout.addWidget(QLabel("X:"), 1, 0)
```

```
    layout.addWidget(QLabel("Y:"), 2, 0)
```

```
    # Valores de posição
```

```
    self.x_pos = QLabel("0.000")
```

```
    self.y_pos = QLabel("0.000")
```

```

pos_font = QFont("Monospace")
pos_font.setPointSize(12)
pos_font.setBold(True)

self.x_pos.setFont(pos_font)
self.y_pos.setFont(pos_font)

self.x_pos.setAlignment(Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter)
self.y_pos.setAlignment(Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter)

layout.addWidget(self.x_pos, 1, 1)
layout.addWidget(self.y_pos, 2, 1)

```

```

def update_position(self, position):
    """Atualiza os valores de posição exibidos"""
    self.x_pos.setText(f'{position.get('x', 0):.3f} mm')
    self.y_pos.setText(f'{position.get('y', 0):.3f} mm')

```

```

class JogController:

```

```

    """Controlador dedicado para operações de jogging"""

```

```

    def __init__(self, comm_thread):
        self.comm_thread = comm_thread
        self.is_jogging = False
        self.current_direction = None
        self.current_axis = None
        self.jog_timer = QTimer()
        self.jog_timer.timeout.connect(self._execute_jog_step)
        self.jog_step_size = 1.0 # Aumentado para 1mm por passo para movimento mais visível
        self.jog_speed = 1000 # mm/min
        self.last_command_time = 0
        self.command_sent = False

```

```

    def start_jog(self, axis, direction):
        """Inicia o modo de jogging contínuo"""
        logger.debug(f'JogController: iniciando jog {axis}{'+' if direction > 0 else '-'}')

        # Se já estiver em jog com os mesmos parâmetros, não reinicia
        if self.is_jogging and self.current_axis == axis and self.current_direction == direction:
            return

        # Para qualquer jog anterior antes de iniciar um novo
        self.stop_jog()

        # Configura o novo movimento
        self.is_jogging = True
        self.current_axis = axis
        self.current_direction = direction
        self.command_sent = False

```

```

# Inicia o processo de jog com base no estado atual da máquina
if "Hold" in self.comm_thread.machine_state:
    logger.debug("JogController: máquina em Hold, enviando comando de retomada")
    self.comm_thread.send_command("~", priority=True)
    # Aguarda para dar tempo ao comando de retomada ser processado
    QTimer.singleShot(100, self._prepare_for_jog)
else:
    self._prepare_for_jog()

def _prepare_for_jog(self):
    """Prepara a máquina para realizar um movimento de jog"""
    # Garante modo relativo
    self.comm_thread.send_command("G91", priority=True)

    # Espera um curto período para garantir que o comando seja processado
    QTimer.singleShot(50, self._start_jog_timer)

def _start_jog_timer(self):
    """Inicia o timer que controla o movimento de jog"""
    if not self.is_jogging:
        return # Verifica se o jog ainda está ativo

    # Executa o primeiro movimento imediatamente
    self._execute_jog_step()

    # Configura o timer para repetir o movimento periodicamente
    # Intervalo maior para melhor estabilidade
    self.jog_timer.start(300) # 300ms entre passos

def _execute_jog_step(self):
    """Executa um único passo de jog"""
    if not self.is_jogging:
        return

    # Impede o envio de comandos muito frequentes
    current_time = time.time()
    if current_time - self.last_command_time < 0.2:
        return

    # Verifica condições de estado da máquina
    if "Hold" in self.comm_thread.machine_state:
        logger.debug("JogController: detectado estado Hold durante movimento, enviando retomada")
        self.comm_thread.send_command("~", priority=True)
        return

    # Calcula a distância do movimento
    distance = self.current_direction * self.jog_step_size

    # Cria o comando de movimento
    if self.current_axis == "X":
        command = f"G0 X{distance} F{self.jog_speed}"

```

```

else: # assume "Y"
    command = f"G0 Y{distance} F{self.jog_speed}"

logger.debug(f"JogController: enviando comando {command}")

```

```

# Envia o comando com alta prioridade
self.comm_thread.send_command(command, priority=True)
self.last_command_time = current_time
self.command_sent = True

```

```

def stop_jog(self):
    """Para o movimento jog"""
    if not self.is_jogging:
        return

```

```

logger.debug("JogController: parando jog")

```

```

# Interrompe o timer
self.jog_timer.stop()
self.is_jogging = False

```

```

# Envia comando de parada apenas se um comando foi enviado antes
if self.command_sent:
    # Envia feed hold para parar movimento
    self.comm_thread.send_command("!", priority=True)

```

```

# Após uma pequena pausa, envia comando de retomada para liberar máquina
QTimer.singleShot(150, self._send_resume_after_stop)

```

```

def _send_resume_after_stop(self):
    """Envia comando de retomada após parada para liberar a máquina"""
    # Envio do comando "~" para liberar máquina do estado Hold
    self.comm_thread.send_command("~", priority=True)

    # Solicita status atualizado
    QTimer.singleShot(50, lambda: self.comm_thread.send_command("?", priority=True))

```

```

class GRBLController(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Controle CNC GRBL Avançado")
        self.setGeometry(100, 100, 1000, 700) # Janela maior para acomodar os controles adicionais

        # Criar thread de comunicação
        self.comm_thread = GRBLCommunicationThread()

        self.jog_controller = JogController(self.comm_thread)

        # Conectar os sinais da thread
        self.comm_thread.response_received.connect(self.on_response_received)
        self.comm_thread.status_update.connect(self.on_status_update)

```

```
self.comm_thread.position_update.connect(self.on_position_update)
self.comm_thread.error_message.connect(self.on_error_message)
```

```
# Iniciar a thread
self.comm_thread.start()
```

```
# Configuração da interface
self.init_ui()
```

```
# Estado da máquina
self.machine_status = "Desconectado"
```

```
# Flag para indicar se estamos em modo de jog
self.jog_mode_active = False
```

```
# Timer para atualizar o status da interface
self.ui_timer = QTimer(self)
self.ui_timer.timeout.connect(self.update_ui_state)
self.ui_timer.start(300) # Atualiza a cada 300ms (mais frequente)
```

```
# Configura a máquina em modo relativo no início
# Isso ajuda a evitar mudanças de modo durante operações de jog
self.current_motion_mode = "G91"
```

```
def init_ui(self):
```

```
    # Widget central
    central_widget = QWidget()
    self.setCentralWidget(central_widget)
```

```
    # Layout principal
    main_layout = QVBoxLayout(central_widget)
```

```
    # Grupo de conexão
    connection_group = QGroupBox("Conexão Arduino")
    connection_layout = QHBoxLayout()
```

```
    self.port_combo = QComboBox()
```

```
    self.connect_button = QPushButton("Conectar")
    self.connect_button.clicked.connect(self.toggle_connection)
```

```
    self.refresh_button = QPushButton("Atualizar Portas")
    self.refresh_button.clicked.connect(self.refresh_ports)
```

```
    connection_layout.addWidget(QLabel("Porta:"))
    connection_layout.addWidget(self.port_combo)
    connection_layout.addWidget(self.connect_button)
    connection_layout.addWidget(self.refresh_button)
    connection_group.setLayout(connection_layout)
```

```
    main_layout.addWidget(connection_group)
```

```

# Layout horizontal para controles e status
control_status_layout = QHBoxLayout()

# Grupo de controle dos eixos (lado esquerdo)
axis_group = QGroupBox("Controle de Eixos")
axis_layout = QGridLayout()

# Controles do eixo X
axis_layout.addWidget(QLabel("Eixo X:"), 0, 0, Qt.AlignmentFlag.AlignRight)

x_control_layout = QHBoxLayout()
self.x_minus_button = JogButton("X-")
self.x_minus_button.pressed.connect(lambda: self.start_jog("X", -1))
self.x_minus_button.released.connect(self.stop_jog)

self.x_plus_button = JogButton("X+")
self.x_plus_button.pressed.connect(lambda: self.start_jog("X", 1))
self.x_plus_button.released.connect(self.stop_jog)

self.x_distance = QLineEdit("1") # Distância reduzida para movimentos mais precisos
self.x_distance.setAlignment(Qt.AlignmentFlag.AlignCenter)

x_control_layout.addWidget(self.x_minus_button)
x_control_layout.addWidget(self.x_distance)
x_control_layout.addWidget(self.x_plus_button)

axis_layout.addLayout(x_control_layout, 0, 1)

# Controles do eixo Y
axis_layout.addWidget(QLabel("Eixo Y:"), 1, 0, Qt.AlignmentFlag.AlignRight)

y_control_layout = QHBoxLayout()
self.y_minus_button = JogButton("Y-")
self.y_minus_button.pressed.connect(lambda: self.start_jog("Y", -1))
self.y_minus_button.released.connect(self.stop_jog)

self.y_plus_button = JogButton("Y+")
self.y_plus_button.pressed.connect(lambda: self.start_jog("Y", 1))
self.y_plus_button.released.connect(self.stop_jog)

self.y_distance = QLineEdit("1") # Distância reduzida para movimentos mais precisos
self.y_distance.setAlignment(Qt.AlignmentFlag.AlignCenter)

y_control_layout.addWidget(self.y_minus_button)
y_control_layout.addWidget(self.y_distance)
y_control_layout.addWidget(self.y_plus_button)

axis_layout.addLayout(y_control_layout, 1, 1)

# NOVO: Controle direcional em formato de teclado numérico (8, 4, 6, 2)

```

```

directional_group = QGroupBox("Controle Direcional")
directional_layout = QGridLayout()

self.dir_up_button = DirectionalButton("up") # 8 no numpad (Y+)
self.dir_left_button = DirectionalButton("left") # 4 no numpad (X-)
self.dir_right_button = DirectionalButton("right") # 6 no numpad (X+)
self.dir_down_button = DirectionalButton("down") # 2 no numpad (Y-)

# Conectar eventos de pressionar/soltar para movimentos imediatos
self.dir_up_button.pressed.connect(lambda: self.start_jog("Y", 1, priority=True))
self.dir_up_button.released.connect(self.stop_jog)

self.dir_down_button.pressed.connect(lambda: self.start_jog("Y", -1, priority=True))
self.dir_down_button.released.connect(self.stop_jog)

self.dir_left_button.pressed.connect(lambda: self.start_jog("X", -1, priority=True))
self.dir_left_button.released.connect(self.stop_jog)

self.dir_right_button.pressed.connect(lambda: self.start_jog("X", 1, priority=True))
self.dir_right_button.released.connect(self.stop_jog)

# Organização em grade para parecer com teclado numérico
directional_layout.addWidget(self.dir_up_button, 0, 1) # Cima (8)
directional_layout.addWidget(self.dir_left_button, 1, 0) # Esquerda (4)
directional_layout.addWidget(self.dir_right_button, 1, 2) # Direita (6)
directional_layout.addWidget(self.dir_down_button, 2, 1) # Baixo (2)

# Jog step - valor incremental para os botões direcionais
step_layout = QHBoxLayout()
step_layout.addWidget(QLabel("Passo:"))
self.jog_step = QLineEdit("1")
self.jog_step.setAlignment(Qt.AlignmentFlag.AlignCenter)
step_layout.addWidget(self.jog_step)
step_layout.addWidget(QLabel("mm"))

directional_layout.addLayout(step_layout, 3, 0, 1, 3)
directional_group.setLayout(directional_layout)

# Adiciona o grupo direcional ao layout principal
axis_layout.addWidget(directional_group, 2, 0, 2, 2)

# Velocidade de movimento (feed rate)
axis_layout.addWidget(QLabel("Velocidade:"), 4, 0, Qt.AlignmentFlag.AlignRight)

feed_layout = QHBoxLayout()
self.feed_rate = QLineEdit("1000") # Valor maior para movimentos mais rápidos
self.feed_rate.setAlignment(Qt.AlignmentFlag.AlignCenter)

# Slider para controle de velocidade
self.feed_slider = QSlider(Qt.Orientation.Horizontal)
self.feed_slider.setMinimum(100)

```

```

self.feed_slider.setMaximum(5000)
self.feed_slider.setValue(1000)
self.feed_slider.setTickPosition(QSlider.TickPosition.TicksBelow)
self.feed_slider.setTickInterval(500)

# Conectar slider e campo de texto
self.feed_slider.valueChanged.connect(self.update_feed_rate)
self.feed_rate.textChanged.connect(self.update_feed_slider)

feed_layout.addWidget(self.feed_rate)
feed_layout.addWidget(QLabel("mm/min"))
feed_layout.addWidget(self.feed_slider)

axis_layout.addLayout(feed_layout, 4, 1)

# Modos de movimento
axis_layout.addWidget(QLabel("Modo:"), 5, 0, Qt.AlignmentFlag.AlignRight)

mode_layout = QHBoxLayout()
self.mode_absolute = QPushButton("Absoluto (G90)")
self.mode_absolute.setCheckable(True)
self.mode_absolute.clicked.connect(lambda: self.set_motion_mode("G90"))

self.mode_relative = QPushButton("Relativo (G91)")
self.mode_relative.setCheckable(True)
self.mode_relative.setChecked(True) # Começa em modo relativo
self.mode_relative.clicked.connect(lambda: self.set_motion_mode("G91"))

mode_layout.addWidget(self.mode_absolute)
mode_layout.addWidget(self.mode_relative)

axis_layout.addLayout(mode_layout, 5, 1)

# Botões de controle da máquina
control_buttons_layout = QGridLayout()

self.home_button = QPushButton("Home ($H)")
self.home_button.clicked.connect(self.home)

self.unlock_button = QPushButton("Desbloquear ($X)")
self.unlock_button.clicked.connect(self.unlock)

self.zero_button = QPushButton("Zerar Posição (G92)")
self.zero_button.clicked.connect(self.set_zero)

self.stop_button = QPushButton("PARAR (!)")
self.stop_button.setStyleSheet("background-color: red; color: white; font-weight: bold;")
self.stop_button.clicked.connect(self.emergency_stop)

control_buttons_layout.addWidget(self.home_button, 0, 0)
control_buttons_layout.addWidget(self.unlock_button, 0, 1)

```

```

control_buttons_layout.addWidget(self.zero_button, 1, 0)
control_buttons_layout.addWidget(self.stop_button, 1, 1)

axis_layout.addLayout(control_buttons_layout, 6, 0, 1, 2)

# Console de comandos
axis_layout.addWidget(QLabel("Comando GCODE:"), 7, 0, 1, 2)

command_layout = QHBoxLayout()
self.command_input = QLineEdit()
self.command_input.returnPressed.connect(self.send_command)

self.send_button = QPushButton("Enviar")
self.send_button.clicked.connect(self.send_command)

command_layout.addWidget(self.command_input)
command_layout.addWidget(self.send_button)

axis_layout.addLayout(command_layout, 8, 0, 1, 2)

axis_group.setLayout(axis_layout)
control_status_layout.addWidget(axis_group)

# Painel de status (lado direito)
status_group = QGroupBox("Status da Máquina")
status_layout = QVBoxLayout()

# Widget de exibição de posição
self.position_display = PositionDisplay()
status_layout.addWidget(self.position_display)

# Status da máquina
status_layout.addWidget(QLabel("Estado:"))
self.machine_state_label = QLabel("Desconectado")
self.machine_state_label.setFont(QFont("Arial", 12, QFont.Weight.Bold))
status_layout.addWidget(self.machine_state_label)

# Últimos comandos e respostas
status_layout.addWidget(QLabel("Último Comando:"))
self.last_command_label = QLabel("-")
status_layout.addWidget(self.last_command_label)

status_layout.addWidget(QLabel("Última Resposta:"))
self.last_response_label = QLabel("-")
status_layout.addWidget(self.last_response_label)

# Log de comunicação
status_layout.addWidget(QLabel("Log de Comunicação:"))
self.comm_log = QLineEdit()
self.comm_log.setReadOnly(True)
status_layout.addWidget(self.comm_log)

```

```

# Otimizações de comunicação
optimization_group = QGroupBox("Otimizações")
optimization_layout = QVBoxLayout()

self.fast_mode_check = QPushButton("Modo Rápido (Sem Door Check)")
self.fast_mode_check.setCheckable(True)
self.fast_mode_check.clicked.connect(self.toggle_fast_mode)
self.fast_mode_check.setToolTip("Desativa verificação de porta para movimentos mais rápidos")

optimization_layout.addWidget(self.fast_mode_check)
optimization_group.setLayout(optimization_layout)
status_layout.addWidget(optimization_group)

status_layout.addStretch()
status_group.setLayout(status_layout)
control_status_layout.addWidget(status_group)

# Proporção entre painéis de controle e status
control_status_layout.setStretch(0, 3) # Controle
control_status_layout.setStretch(1, 2) # Status

main_layout.addLayout(control_status_layout)

# Barra de status
self.statusBar = self.statusBar()
self.statusBar.showMessage("Pronto para conexão")

# Desabilitar controles até que a conexão seja estabelecida
self.toggle_controls(False)

# Atualizar portas
self.refresh_ports()

def _resume_after_stop(self):
    """Método auxiliar para enviar comando de retomada após parada"""
    try:
        # Verifica se a máquina está em estado Hold e envia comando para retomar
        if "Hold" in self.comm_thread.machine_state:
            logger.debug("Enviando comando de retomada após parada")
            self.comm_thread.send_command("~", priority=True)

        # Solicita status para atualizar interface
        self.comm_thread.send_command("?", priority=True)
    except Exception as e:
        logger.error(f"Erro ao retomar após parada: {e}", exc_info=True)

def refresh_ports(self):
    """Detecta automaticamente as portas seriais disponíveis"""
    self.port_combo.clear()

```

```

available_ports = [port.device for port in serial.tools.list_ports.comports()]

if not available_ports:
    self.statusBar.showMessage("Nenhuma porta serial encontrada")
    return

self.port_combo.addItems(available_ports)

# Verificar se COM9 está na lista e selecioná-la
com9_index = self.port_combo.findText("COM9")
if com9_index >= 0:
    self.port_combo.setCurrentIndex(com9_index)
    self.statusBar.showMessage("Arduino detectado na porta COM9")
else:
    self.statusBar.showMessage(f"Portas encontradas: {' '.join(available_ports)}")

def toggle_connection(self):
    if not self.comm_thread.is_connected:
        try:
            port = self.port_combo.currentText()
            if not port:
                self.statusBar.showMessage("Nenhuma porta selecionada")
                return

            # Mensagem de status
            self.statusBar.showMessage(f"Conectando à porta {port}...")

            # Configuração de porta similar ao consumo_lib.py
            serial_port = serial.Serial(port, 115200, timeout=0.5)
            time.sleep(0.5) # Aguarda a inicialização básica

            # Inicializar GRBL - similar ao consumo_lib.py
            logger.info("Enviando comando de inicialização")
            serial_port.write(b"\r\n\r\n")
            time.sleep(0.5)

            # Limpar buffer
            logger.info("Limpendo buffer de entrada")
            serial_port.flushInput() # Usa flushInput como em consumo_lib

            # Configurar a thread
            logger.info("Configurando thread de comunicação")
            self.comm_thread.set_serial_port(serial_port)

            # Enviar comandos de configuração
            logger.info("Enviando comando de desbloqueio")
            self.comm_thread.send_command("$X", priority=True)
            time.sleep(0.1) # Pequeno delay para processamento do comando

            # Iniciar em modo relativo para jog
            logger.info("Configurando modo relativo")

```

```
self.comm_thread.send_command("G91", priority=True)
self.mode_relative.setChecked(True)
self.mode_absolute.setChecked(False)

# Confirmar conexão bem-sucedida
logger.info("Conexão estabelecida com sucesso")
self.connect_button.setText("Desconectar")
self.machine_state_label.setText("Conectado")
self.statusBar.showMessage(f"Conectado ao GRBL na porta {port}")
self.toggle_controls(True)
```

```
except Exception as e:
```

```
    logger.error(f"Erro ao conectar: {str(e)}", exc_info=True)
    self.statusBar.showMessage(f"Erro: {str(e)}")
```

```
else:
```

```
    # Desconectar
    logger.info("Desconectando")
    self.comm_thread.disconnect()
```

```
    self.connect_button.setText("Conectar")
    self.machine_state_label.setText("Desconectado")
    self.statusBar.showMessage("Desconectado")
    self.toggle_controls(False)
```

```
def toggle_controls(self, enabled):
```

```
    """Habilita/desabilita os controles com base no estado de conexão"""
```

```
    # Controles de eixos e botões de movimento
    self.x_minus_button.setEnabled(enabled)
    self.x_plus_button.setEnabled(enabled)
    self.y_minus_button.setEnabled(enabled)
    self.y_plus_button.setEnabled(enabled)
```

```
    # Botões direcionais
    self.dir_up_button.setEnabled(enabled)
    self.dir_down_button.setEnabled(enabled)
    self.dir_left_button.setEnabled(enabled)
    self.dir_right_button.setEnabled(enabled)
    self.jog_step.setEnabled(enabled)
```

```
    # Campos de entrada
    self.x_distance.setEnabled(enabled)
    self.y_distance.setEnabled(enabled)
    self.feed_rate.setEnabled(enabled)
    self.feed_slider.setEnabled(enabled)
```

```
    # Botões de controle
    self.home_button.setEnabled(enabled)
    self.unlock_button.setEnabled(enabled)
    self.zero_button.setEnabled(enabled)
    self.stop_button.setEnabled(enabled)
    self.fast_mode_check.setEnabled(enabled)
```

```

# Modos de movimento
self.mode_absolute.setEnabled(enabled)
self.mode_relative.setEnabled(enabled)

# Console de comandos
self.command_input.setEnabled(enabled)
self.send_button.setEnabled(enabled)

def update_ui_state(self):
    """Atualiza o estado da interface com base no status da máquina"""
    # Colorir o status da máquina
    if self.machine_status == "Idle":
        self.machine_state_label.setStyleSheet("color: green;")
    elif self.machine_status == "Run":
        self.machine_state_label.setStyleSheet("color: blue;")
    elif self.machine_status == "Alarm":
        self.machine_state_label.setStyleSheet("color: red;")
    elif self.machine_status == "Hold":
        self.machine_state_label.setStyleSheet("color: orange;")
    else:
        self.machine_state_label.setStyleSheet("")

def toggle_fast_mode(self):
    """Ativa/desativa o modo rápido (desabilita verificação de porta)"""
    if self.fast_mode_check.isChecked():
        # Desativar verificação de porta para movimentos mais rápidos
        self.comm_thread.send_command("$I0=0", priority=True)
        self.statusBar.showMessage("Modo rápido ativado (verificação de porta desativada)")
    else:
        # Reativar verificação de porta
        self.comm_thread.send_command("$I0=1", priority=True)
        self.statusBar.showMessage("Modo rápido desativado (verificação de porta ativada)")

def set_motion_mode(self, mode):
    """Define o modo de movimento (absoluto ou relativo)"""
    if mode == "G90":
        self.mode_absolute.setChecked(True)
        self.mode_relative.setChecked(False)
    else:
        self.mode_absolute.setChecked(False)
        self.mode_relative.setChecked(True)

# Atualiza o modo atual
self.current_motion_mode = mode

# Enviar comando para a máquina
self.comm_thread.send_command(mode)
self.last_command_label.setText(mode)

def update_feed_rate(self, value):

```

```
"""Atualiza o campo de texto quando o slider muda"""
self.feed_rate.setText(str(value))
```

```
def update_feed_slider(self):
    """Atualiza o slider quando o campo de texto muda"""
    try:
        value = int(self.feed_rate.text())
        # Limitar ao intervalo do slider
        value = max(100, min(5000, value))
        self.feed_slider.setValue(value)
    except ValueError:
        pass
```

```
def start_jog(self, axis, direction, priority=False):
    """Inicia movimento jog para um eixo"""
    try:
        logger.debug(f"Iniciando jog: eixo={axis}, direção={direction}")

        # Delega o jogging para o controlador dedicado
        self.jog_controller.start_jog(axis, direction)

    except Exception as e:
        logger.error(f"Erro ao iniciar movimento: {e}", exc_info=True)
        self.statusBar.showMessage(f"Erro ao iniciar movimento: {str(e)}")
```

```
def _continue_jog(self, axis, direction, priority):
    """Continua o processo de jog após verificações iniciais"""
    try:
        # Determina a distância de movimento
        if axis == "X":
            distance = float(self.x_distance.text())
        elif axis == "Y":
            distance = float(self.y_distance.text())
        else:
            # Para outros casos, usa o jog_step
            distance = float(self.jog_step.text())

        # Aplica direção
        if direction < 0:
            distance = -distance

        # Obtém a velocidade
        feed_rate = float(self.feed_rate.text())

        logger.debug(f"Parâmetros de jog: distância={distance}, velocidade={feed_rate}")

        # Garante que estamos em modo relativo para jog - sem bloquear a UI
        if self.current_motion_mode != "G91":
            logger.debug("Mudando para modo relativo (G91)")
            self.comm_thread.send_command("G91", priority=True)
            self.current_motion_mode = "G91"
```

```

        self.mode_relative.setChecked(True)
        self.mode_absolute.setChecked(False)

    # Monta o comando G-code
    g_command = "G0" if priority else "G1"

    if axis == "X":
        gcode = f"{g_command} X{distance} F{feed_rate}"
    else:
        gcode = f"{g_command} Y{distance} F{feed_rate}"

    logger.info(f"Enviando comando jog: {gcode}")

    # Envia o comando
    self.comm_thread.send_command(gcode, priority=priority)
    self.last_command_label.setText(gcode)
except Exception as e:
    logger.error(f"Erro ao continuar movimento: {e}", exc_info=True)
    self.statusBar.showMessage(f"Erro ao continuar movimento: {str(e)}")

def stop_jog(self):
    """Para o movimento jog quando o botão é liberado"""
    try:
        logger.debug("Parando movimento jog")

        # Delega para o controlador
        self.jog_controller.stop_jog()

    except Exception as e:
        logger.error(f"Erro ao parar movimento: {e}", exc_info=True)
        self.statusBar.showMessage(f"Erro ao parar movimento: {str(e)}")

def home(self):
    """Envia comando para ir para home"""
    self.comm_thread.send_command("$H")
    self.last_command_label.setText("$H (Home)")

def unlock(self):
    """Desbloqueia a máquina após um alarme"""
    self.comm_thread.send_command("$X", priority=True)
    self.last_command_label.setText("$X (Unlock)")

def set_zero(self):
    """Define a posição atual como zero para todos os eixos"""
    self.comm_thread.send_command("G92 X0 Y0")
    self.last_command_label.setText("G92 X0 Y0 (Set Zero)")

def emergency_stop(self):
    """Para todos os movimentos imediatamente"""
    self.comm_thread.send_command("!", priority=True)
    self.statusBar.showMessage("PARADA DE EMERGÊNCIA acionada!")

```

```
self.last_command_label.setText("! (Parada de Emergência)")
```

```
def send_command(self):
```

```
    """Envia um comando G-code personalizado"""
```

```
    command = self.command_input.text()
```

```
    if command:
```

```
        self.comm_thread.send_command(command)
```

```
        self.last_command_label.setText(command)
```

```
        self.command_input.clear()
```

```
def on_response_received(self, response):
```

```
    """Manipula a resposta recebida da thread de comunicação"""
```

```
    self.last_response_label.setText(response)
```

```
    self.comm_log.setText(response)
```

```
def on_status_update(self, status):
```

```
    """Atualiza o status da máquina na interface"""
```

```
    # Extrai o estado da máquina (ex: <Idle|...> => "Idle")
```

```
    if status.startswith('<') and '|' in status:
```

```
        state = status[1:status.find('|')]
```

```
        self.machine_status = state
```

```
        self.machine_state_label.setText(state)
```

```
def on_position_update(self, position):
```

```
    """Atualiza a exibição de posição na interface"""
```

```
    self.position_display.update_position(position)
```

```
def on_error_message(self, error):
```

```
    """Exibe mensagem de erro na barra de status"""
```

```
    self.statusBar.showMessage(error)
```

```
def closeEvent(self, event):
```

```
    """Manipula o evento de fechamento da janela"""
```

```
    self.comm_thread.running = False # Sinaliza que a thread deve terminar
```

```
    self.comm_thread.wait(1000)      # Espera até 1 segundo pela thread
```

```
    event.accept()
```

```
if __name__ == "__main__":
```

```
    app = QApplication(sys.argv)
```

```
    window = GRBLController()
```

```
    window.show()
```

```
    sys.exit(app.exec())
```